BECE355L — Advanced Cloud Computing

Cloud-Native Parallel Image Processing

How to Run — Detailed Guidebook

Setup from a fresh clone, the three ways to run the framework (local, Docker Compose, Kubernetes), the web UI and REST API, the cost-aware planner, the work-aware autoscaler and its baselines, and how to reproduce every result in the report.

Name	Register No.
Prapti Kurkuri	24BVD0185
Vaanya Garg	24BVD0139
Aditya Manoj	24BVD0070
Arrnav Trisal	24BVD0064

Repository: github.com/GHOSTALERT/cloud-native-ing-proc

Contents

1. What you are running
2. Prerequisites
3. One-time setup
4. Mode A: local run (no Docker)
5. Using the web UI
6. Using the REST API
7. Mode B: Docker Compose
8. Mode C: Kubernetes (Minikube)
9. Autoscalers: work-aware, KEDA and HPA
10. Reproducing every result in the report
11. Deploying on AWS (S3, IAM, KMS, EKS)
12. Configuration reference
13. Troubleshooting

1. What you are running

A large image is uploaded to a REST API. A **cost-aware planner** predicts how long the job will take locally and distributed, and picks the faster plan, the tile size and the worker count. The image is cut into tiles with overlapping halos; the tiles are stored in **S3-compatible object storage** (MinIO locally, Amazon S3 on AWS) and only their ids go into a **Redis** task queue. **Worker** processes or pods claim tiles with a lease, run an OpenCV filter or a neural network (HED edges, Real-ESRGAN 4× super-resolution), store the result in object storage, and the worker that finishes the last tile stitches the final image. A **work-aware autoscaler** sizes the worker Deployment from the predicted compute-seconds waiting in the queue.

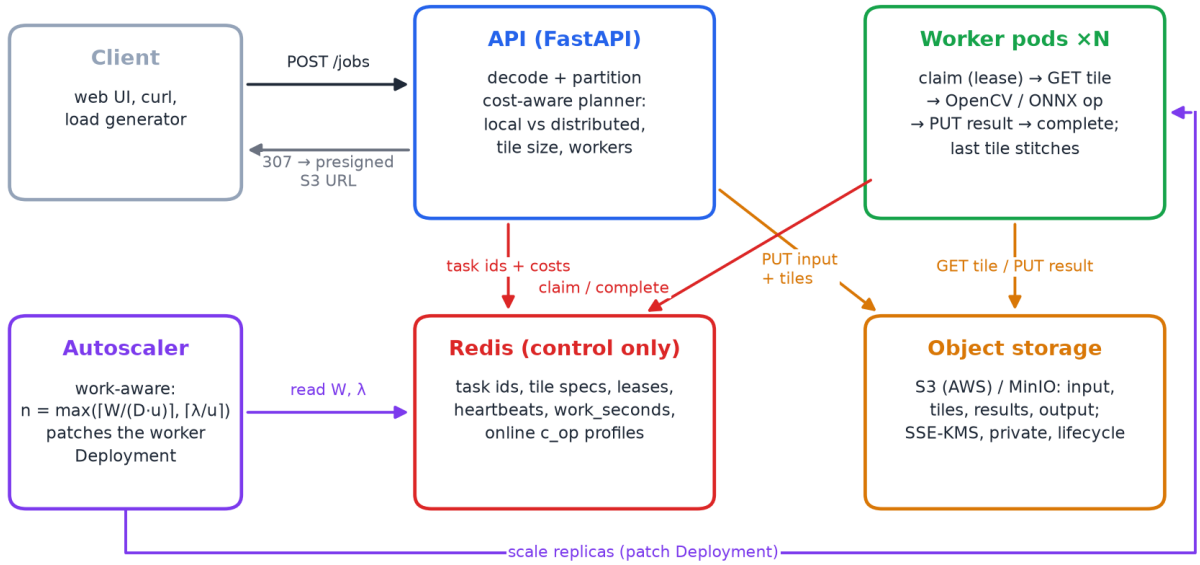


Figure 1. System architecture after the four upgrades.

Table 1. Repository layout

Folder	What is inside
imgframe/	core library: config, ops (OpenCV + ONNX), tiling, Redis store + queue, object store, planner, models
api/	FastAPI service and the web UI (index.html)
worker/	worker process (claim → fetch → process → store → aggregate)
autoscaler/	work-aware / queue-length autoscaler for Kubernetes
benchmark/	benchmarks; results in benchmark/results/
scripts/	run_local, model and MinIO fetchers, calibration, profiling, load tests, K8s helpers
k8s/	Kubernetes manifests; optional/ holds the HPA and KEDA baselines
deploy/aws/	S3 bucket policy, IAM policies, lifecycle rule, KMS setup script, IRSA service accounts
docs/	report, slides, this guide, figures and the scripts that build them
tests/	pytest suite (43 tests)

2. Prerequisites

Everything was developed and tested on Ubuntu (x86-64, 16 hardware threads, 22 GB RAM). Any recent Linux works; macOS works for modes A and B.

Table 2. Software prerequisites

Software	Needed for	How to get it
Python 3.12 or 3.13 + venv	everything	sudo apt install python3 python3-venv
git, curl, gcc, make	cloning, downloads, building	sudo apt install git curl build-essential

	Redis	
Docker	modes B and C, and fetching the MinIO binary	sudo bash scripts/install_k8s_tools.sh
Minikube + kubectl	mode C	same script (installs Docker, Minikube v1.37, kubectl v1.34)
LibreOffice	only to rebuild the PDF documents	sudo apt install libreoffice
Firefox	only to recapture UI screenshots	preinstalled on Ubuntu

Tip: Nothing in modes A and B needs root. Redis and MinIO run from .tools/bin inside the project.

3. One-time setup

3.1 Clone and create the Python environment

```
git clone https://github.com/GHOSTALERT/cloud-native-ing-proc.git
cd cloud-native-ing-proc
python3 -m venv .venv
.venv/bin/pip install -r requirements-dev.txt
```

If python3 -m venv fails because ensurepip is missing (Debian/Ubuntu without python3-venv), create the venv without pip and let the system pip install into it:

```
python3 -m venv --without-pip .venv
pip3 --python .venv/bin/python install -r requirements-dev.txt
```

3.2 Redis and MinIO binaries (for mode A)

```
bash scripts/build_redis.sh      # compiles redis-server 7.2.5 into .tools/bin (about 1 minute)
bash scripts/fetch_minio.sh      # copies the MinIO server binary into .tools/bin (needs Docker)
```

MinIO no longer publishes binaries or Docker Hub images, so fetch_minio.sh copies the binary out of Chainguard's open-source build (cgr.dev/chainguard/minio). Without .tools/bin/minio, run_local.py falls back to the original all-in-Redis storage mode, so the framework still runs.

3.3 Neural network weights

```
bash scripts/fetch_models.sh     # ~125 MB download, checksums verified
```

This downloads the official HED Caffe model (Xie & Tu) and the Real-ESRGAN general-x4v3 ONNX export, and converts HED to models/hed.onnx with scripts/convert_hed.py (OpenCV 5 dropped its Caffe importer, so the converter decodes the caffemodel itself and rebuilds the network for ONNX Runtime). Without the weights the hed and superres operations are simply hidden from the UI and /ops.

3.4 Sample images and the test suite

```
.venv/bin/python scripts/make_samples.py     # samples/astronaut_2048.png, synthetic_8192.png
.venv/bin/python -m pytest -q                # 43 passed
```



Figure 2. The test suite: 43 tests, including S3 mode, neural tiling, planner and work accounting.

4. Mode A: local run (no Docker)

scripts/run_local.py starts Redis, MinIO, the API and N worker processes on this machine:

```
.venv/bin/python scripts/run_local.py --workers 4
# API ready at http://127.0.0.1:8000  (4 workers, redis ..., storage MinIO http://127.0.0.1:9000)
```

Open **http://127.0.0.1:8000** in a browser. Stop everything with Ctrl+C.

4.1 Scripted demos

```
# one job, checked pixel-by-pixel against whole-image processing
.venv/bin/python scripts/run_local.py --workers 4 --demo --op bilateral --size 4096


# neural edge detection on the sample photo, planner decides the tiling
.venv/bin/python scripts/run_local.py --workers 8 --demo --op hed --image samples/astronaut_2048.png


# 4x super-resolution of a 512 px image (output 2048 px)
.venv/bin/python scripts/run_local.py --workers 4 --demo --op superres --size 512


# fault tolerance: SIGKILL a worker mid-job; its tiles are retried, the result is still exact
.venv/bin/python scripts/run_local.py --workers 4 --demo --kill-worker --op bilateral --tile-size 256
```

Table 3. run_local.py options

Option	Meaning
--workers N	number of worker processes (default 4)
--storage s3|redis	s3 (default when MinIO is present): pixels in MinIO; redis: original mode
--tile-size auto|N	auto lets the planner decide; a number forces naive tiling with that size
--op NAME	canny, sobel, gaussian_blur, sharpen, grayscale, bilateral, hed, superres
--image PATH / --size N	input image for --demo, else a generated N×N image
--port, --redis-port, --minio-port	change the ports (8000, 6390, 9000) if they are in use

Tip: The demo's pixel-agreement line is the correctness check: 100 % for local filters, ≥ 99.9 % for Canny, and ≥ 40 dB seam PSNR for the neural ops at their calibrated halos.

5. Using the web UI

1) Choose an image (PNG/JPEG/TIFF) and an operation.
2) Leave **Tile size** on auto to let the planner choose local vs distributed and the tile size, or type a number (256, 512, ...) to force naive tiling.
3) Press **Process**. The progress bar counts finished tiles; the right-hand panel shows the plan, the storage mode, per-phase timings, and the planner's predicted time next to the measured one.
4) When the job is done, the result appears next to the original. Full-resolution links are short-lived presigned S3 URLs (5 minutes by default); huge outputs are offered as a tiled TIFF.
5) The **Cluster** card shows queued work in compute-seconds (the autoscaler's input), pending and in-progress tiles, retries, live workers and Redis memory.

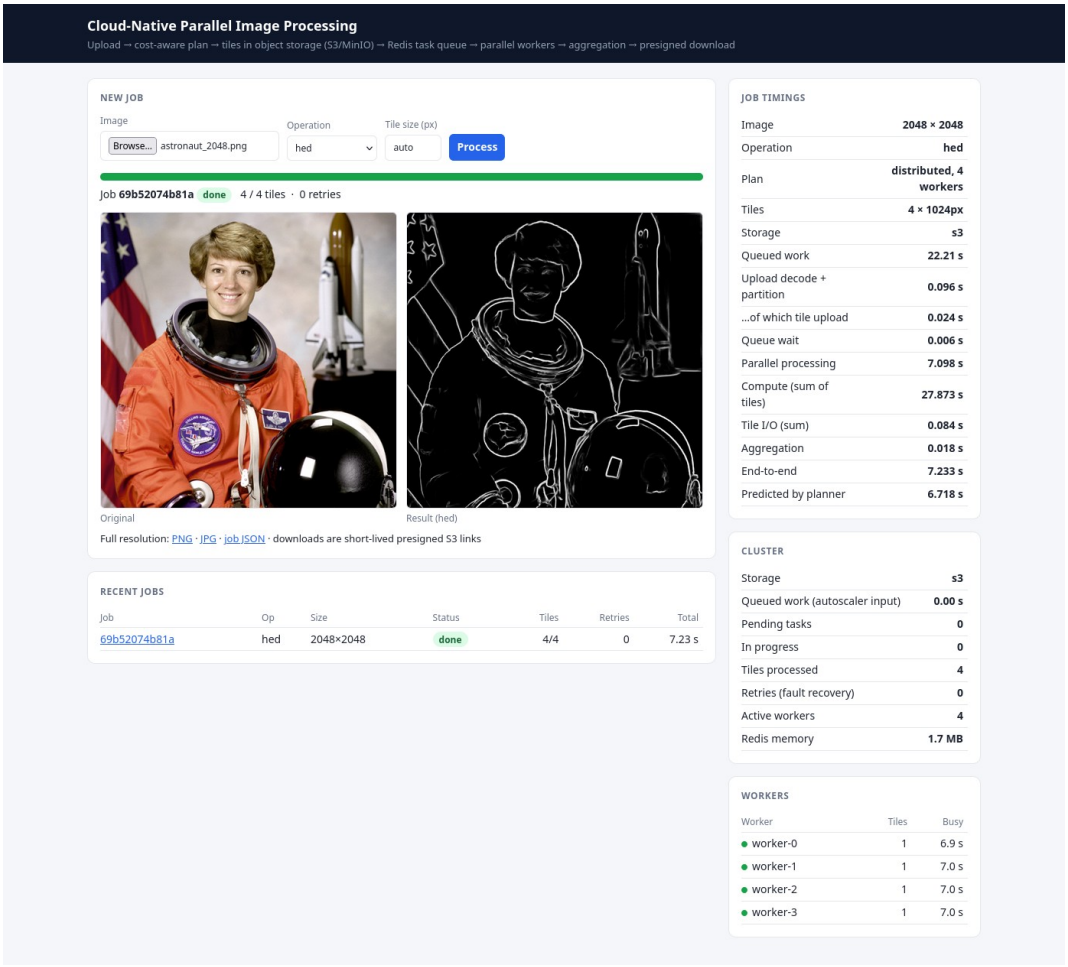


Figure 3. Web UI after an HED job planned automatically.

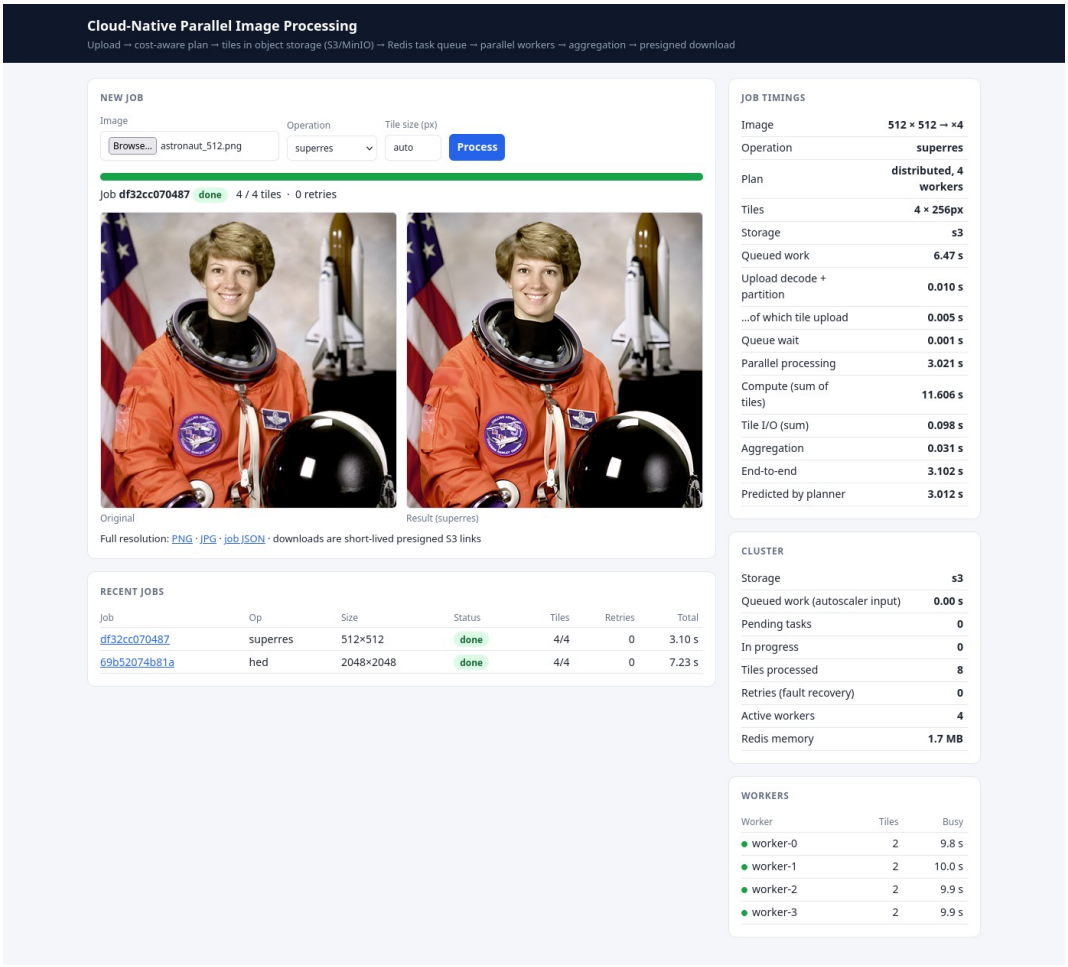


Figure 4. 4× super-resolution job: the plan, scale factor and presigned download.

6. Using the REST API

Table 4. REST endpoints

Method + path	Purpose
POST /jobs (form: file, op, tile_size=auto|N)	submit an image; returns the job with its plan
GET /jobs, GET /jobs/{id}	list jobs / status, progress, plan, timings
GET /jobs/{id}/result?format=png|jpg|webp|tif&max_side=N	final image (S3 mode: 307 redirect to a presigned URL)
GET /jobs/{id}/url?format=...	the presigned URL as JSON, with its expiry
GET /jobs/{id}/preview	downscaled copy of the upload
DELETE /jobs/{id}	delete the job and its objects
GET /plan?op=&width=&height=&workers=	dry run of the planner (nothing is queued)
GET /metrics	queue depth, queued work-seconds, workers, retries, Redis memory
GET /ops, GET /healthz, GET /docs	operations, health check, Swagger UI

```
API=http://127.0.0.1:8000
curl -s -F file=@samples/astronaut_2048.png -F op=hed $API/jobs | python3 -m json.tool
curl -s $API/jobs/<id> | python3 -m json.tool        # status, plan_mode, predicted_s, timings
curl -sL -o edges.png $API/jobs/<id>/result          # -L follows the presigned redirect
curl -s "$API/plan?op=canny&width=8192&height=8192"   # why is canny run locally?
curl -s $API/metrics | python3 -m json.tool
```

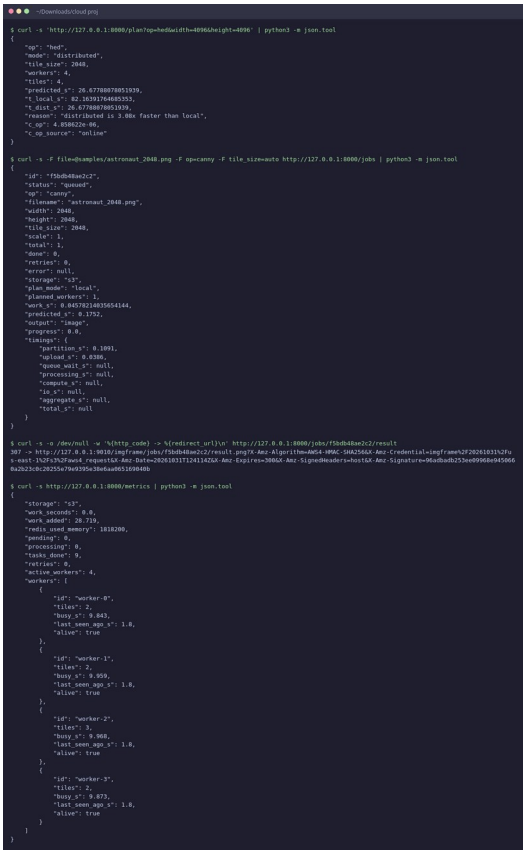


Figure 5. Submitting a job and asking the planner with curl.

7. Mode B: Docker Compose

Compose runs Redis (capped at 256 MB: it only holds control data now), MinIO, the API and four workers from one image. Fetch the models first; they are baked into the image.

```
bash scripts/fetch_models.sh
docker compose up --build --scale worker=4
# UI http://localhost:8000      MinIO (presigned downloads) http://localhost:9000
bash scripts/kill_worker.sh compose      # optional: crash a busy worker during a job
docker compose down
```

8. Mode C: Kubernetes (Minikube)

8.1 Start the cluster and build the image inside it

```
minikube start --driver=docker --cpus=8 --memory=12g
eval $(minikube docker-env)
docker build -t imgframe:latest .
```

8.2 Deploy

```
kubectl apply -f k8s/
kubectl -n imgframe set env deploy/api S3_PUBLIC_ENDPOINT=http://$(minikube ip):30900
kubectl -n imgframe get pods -w          # redis, minio, api, autoscaler, worker(s) Running
echo http://$(minikube ip):30080         # the web UI
```

The second command matters: presigned URLs are signed for a host name, and the browser reaches MinIO through the NodePort 30900 on the Minikube IP.

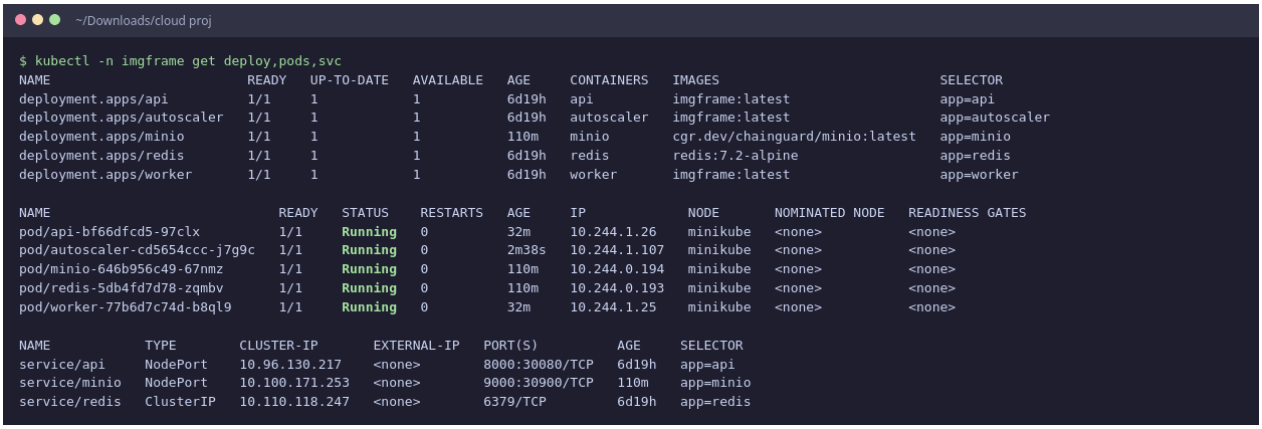


Figure 6. All components running on Minikube.

Table 5. Kubernetes manifests

Manifest	Creates
00-namespace.yaml	namespace, ConfigMap (Redis/S3 settings, PLANNER_WORKERS=8), demo S3 credentials
10-redis.yaml	Redis 7.2, 256 MB, control data only
15-minio.yaml	MinIO object store, NodePort 30900
20-api.yaml	API Deployment + NodePort 30080
30-worker.yaml	worker Deployment (1 CPU, 2 GiB limit each)
40-autoscaler.yaml	work-aware autoscaler + RBAC to scale the worker Deployment
optional/hpa-cpu.yaml	baseline: CPU-based HorizontalPodAutoscaler (70 %)
optional/keda-scaledobject.yaml	baseline: KEDA scaling on Redis list length

8.3 Demos on the cluster

```
API=http://$(minikube ip):30080
.venv/bin/python scripts/load_test.py --api $API --jobs 6 --op bilateral --size 4096
kubectl -n imgframe logs -f deploy/autoscaler          # scaling decisions with W and λ
bash scripts/kill_worker.sh k8s                    # self-healing: crash a busy worker pod
```

9. Autoscalers: work-aware, KEDA and HPA

Exactly one scaler may own the worker Deployment. Install the two baselines once, then switch with the helper script:

```
bash scripts/install_k8s_addons.sh     # metrics-server addon + KEDA 2.21 (one time)
bash scripts/k8s_scaler.sh work       # ours: replicas from queued compute-seconds
bash scripts/k8s_scaler.sh keda       # KEDA: 16 pending tiles per worker
bash scripts/k8s_scaler.sh hpa        # HPA: 70 % CPU
bash scripts/k8s_scaler.sh queue      # our autoscaler with the original queue-length rule
```

The work-aware rule is replicas = clamp(max(⌈W/(D·u)⌉, ⌈λ/u⌉), MIN, MAX) with W the predicted seconds of queued work, λ its arrival rate, D the latency target (LATENCY_TARGET_S minus POD_STARTUP_S) and u the target utilization. Tune it with kubectl -n imgframe set env deploy/autoscaler LATENCY_TARGET_S=20.

9.1 Replaying the comparison trace

```
bash scripts/run_autoscale_comparison.sh 3     # ~45 min: 3 runs x (hpa, keda, work), interleaved


# or one run by hand:
bash scripts/k8s_scaler.sh keda
.venv/bin/python scripts/trace_replay.py --api http://$(minikube ip):30080 --label keda --run 1
.venv/bin/python docs/make_figures.py autoscale
```

The driver switches the scaler, waits until the queue is empty and the Deployment is back at one ready worker, waits 20 s more so HPA and KEDA see an idle sample, and then replays the

trace. Runs are interleaved (hpa, keda, work, hpa, ...) so slow drift on the host affects all three alike.

Each run replays the same seeded Poisson trace (bursts of cheap grayscale/Canny jobs, then bursts of bilateral/HED jobs) and writes benchmark/results/autoscale/<scaler>_run<N>.json with per-job latency, p50/p95, jobs within the target, worker pod-seconds, scaling events and time to first scale-up.

10. Reproducing every result in the report

Table 6. Commands that regenerate each result (run from the repository root with .venv/bin/python)

Result	Command	Output in benchmark/results/
Halo calibration (HED, Real-ESRGAN)	python scripts/calibrate_halo.py	halo_calibration.json/.png; imgframe/calibration.json
Cost-model profiles	python scripts/profile_ops.py	imgframe/profiles.json
Original naive-tiling benchmark	python -m benchmark.bench_local --size 8192	results.csv, charts.png
Planner vs naive, model accuracy	python -m benchmark.bench_planner	planner_engine.json/.png
Planner on the full framework	python -m benchmark.bench_planner --cluster --workers 8	planner_cluster.json
Redis vs S3 storage study	python -m benchmark.bench_storage	storage_study.json
Autoscaler comparison (Minikube)	bash scripts/run_autoscale_comparison.sh 3	autoscale/<scaler>_run<N>.json
Screenshots	python scripts/capture_screenshots.py --k8s	docs/screenshots/_*v2*.png
All report figures	python docs/make_figures.py	docs/figures/*.png
Report, slides, this guide	python docs/build_report.py; python docs/build_slides.py; python docs/build_guide.py	docs/*.docx, .pdf, .pptx

Watch out: Benchmarks measure wall-clock time: close other heavy programs and do not run two benchmarks at once. Numbers will differ on other hardware; the planner adapts because it reads profiles.json, so re-run profile_ops.py on a new machine first.

11. Deploying on AWS (S3, IAM, KMS, EKS)

The deploy/aws/ templates implement the security controls from Module 4. They were written for this project but **not executed** (no AWS account was used); review them before running.

1) Create the bucket, a customer-managed KMS key, default SSE-KMS encryption, Block Public Access, a bucket policy that denies unencrypted or non-TLS requests, the lifecycle rule and two least-privilege IAM policies: BUCKET=my-imgframe REGION=ap-south-1 bash deploy/aws/setup_s3.sh.
2) Create IAM roles imgframe-api and imgframe-worker with those policies and trust for EKS IRSA; apply deploy/aws/irsa-serviceaccounts.yaml and add serviceAccountName to the api and worker Deployments.
3) In the ConfigMap: remove S3_ENDPOINT, set S3_BUCKET, S3_SSE=aws:kms, S3_KMS_KEY_ID and S3_MANAGE_BUCKET=0; delete the s3-credentials Secret and the MinIO manifest.
4) Push the image to ECR, change imagePullPolicy to IfNotPresent and apply the remaining manifests.

Table 7. Least-privilege split between the two service roles

Role	May do	Cannot do
API	PUT jobs/*/input.*, tiles/*, result.*; GET outputs; delete jobs	read tiles or results; touch other prefixes

Worker	GET tiles/results; PUT results and outputs; delete tiles/results after stitching	write inputs; delete outputs; list the bucket outside jobs/

12. Configuration reference

Table 8. Environment variables

Variable	Default	Meaning
REDIS_URL	redis://localhost:6379/0	Redis connection
S3_BUCKET	(empty)	set → object-storage mode; empty → all-in-Redis mode
S3_ENDPOINT / S3_PUBLIC_ENDPOINT	(AWS) / = endpoint	S3 API for the services / host used in presigned URLs
S3_SSE, S3_KMS_KEY_ID	(none)	server-side encryption (aws:kms + key ARN)
S3_MANAGE_BUCKET	1	create bucket + lifecycle at start (MinIO); 0 on AWS
PRESIGN_SECONDS	300	lifetime of download URLs
RESULT_DELIVERY	redirect	redirect (307 to S3) or proxy (stream through the API)
MAX_QUEUED_TILES	20000	back-pressure in S3 mode (HTTP 503 beyond it)
STREAM_OUTPUT_MP	150	outputs above this many megapixels are streamed to a tiled TIFF
MODEL_DIR	./models	ONNX weights
PLANNER, PLANNER_MARGIN	1, 0.2	planner on/off; distribute only if ≥ 20 % faster
PLANNER_WORKERS	0 (= live workers)	workers the planner may assume (8 on Kubernetes)
PROFILE_EMA	0.05	weight of each new tile in the online c_op estimate
TILE_SIZE	512	tile size when the planner is off
LEASE_SECONDS, HEARTBEAT_SECONDS, MAX_RETRIES	15, 3, 3	fault-tolerance timing
WORKER_PROCS, OPENCV_THREADS	1, 1	process slots per worker; threads per op (also ONNX Runtime)
CHAOS_CRASH_RATE	0	probability a worker crashes mid-tile (fault injection)
POLICY	work	autoscaler rule: work or queue
LATENCY_TARGET_S, TARGET_UTILIZATION, POD_STARTUP_S	30, 0.8, 0	work-aware rule parameters
MIN_WORKERS, MAX_WORKERS, SCALE_DOWN_DELAY	1, 8, 30	autoscaler bounds and scale-down delay
IDLE_SCALE_DOWN_S	10	work mode: once nothing is queued or running for this long, drop to MIN_WORKERS

13. Troubleshooting

Table 9. Common problems

Symptom	Cause and fix
Port 8000 already in use	another run_local.py or Compose stack is running: stop it, or pass --port, --redis-port, --minio-port
.tools/bin/minio missing	run bash scripts/fetch_minio.sh, or use --storage redis
hed/superres missing from the list	weights not downloaded: bash scripts/fetch_models.sh

HTTP 503 on upload	back-pressure: the queue (S3 mode) or Redis memory (Redis mode) is full; the client retries after Retry-After
Result link fails in the browser (Kubernetes)	presigned host is wrong: set S3_PUBLIC_ENDPOINT=http://$(minikube ip):30900 on the api
Pods stuck in ErrImageNeverPull	image built outside Minikube: run eval $(minikube docker-env) and rebuild
HPA shows <unknown> CPU	metrics-server not ready: minikube addons enable metrics-server, wait a minute
no matches for kind ScaledObject	KEDA not installed: bash scripts/install_k8s_addons.sh
Worker pod OOMKilled on HED	raise the worker memory limit in k8s/30-worker.yaml or use 512 px tiles
Benchmarks much slower than the report	other load on the machine, or different hardware: re-run scripts/profile_ops.py